BinaryTree-master

340/BinaryTree-master/README.md

CSCI 340 - Data Structures and Algorithm Analysis

Data Structure: Binary Tree
Programming Focus: C++ class development

Binary Tree Class

For this computer assignment, you are to write a C++ program to implement a class for binary trees. To deal with variety of data types, implement this class as a template. The definition of the class for a binary tree (as a template) is given as follows:

```
        template <typename T> class BinaryTree{

        public:
            BinaryTree();                                   // default constructor
            unsigned     getSize() const;                   // returns size of tree
            unsigned     getHeight() const;                 // returns height of tree
            virtual void Insert(const T&);                  // inserts node in tree
            void         Inorder(void (*)(const T&));       // inorder traversal of tree

        protected:
            Node<T> *root;                                   // root of tree

        private:
            unsigned _getSize(Node<T> *) const;              // private version of getSize()
            unsigned _getHeight(Node<T> *) const;            // private version of
getHeight()
            void     _Insert(Node<T> *&, const T&);          // private version of Insert()
            void     _Inorder(Node<T> *, void (*)(const T&));   // private version of Inorder()
        };
```

Most of the *public* member functions of the `BinaryTree` class call *private* member functions of the class (with the same name). These *private* member functions can be implemented as either `recursive` or *non-recursive*, but clearly, *recursive* versions of these functions are preferable because of their short and simple implementations in code. However, they require more memory and usually slower than their *non-recursive* versions in execution, especially for a large amount of input data.

Because of information hiding, a client is not permitted to access the binary tree directly, so the *root* of the tree is kept protected (not *private* because of future implementations of derived classes from the base class of the `BinaryTree`), so it cannot be passed as an argument to any of the *public* functions of the tree. It is essential to have *private* utility functions, which act as interface between a client and the tree. The `Insert()` function of the `BinaryTree` class is described as follows:

- `Insert(const T &x)` This *virtual* function can be used to insert a node with the data value `x` in a binary tree, applying the following technique: if the tree is empty, then the new node will be the root of the tree with the value `x`; otherwise, the left or the right subtree is **randomly** selected and the value `x` is inserted in that side. To implement the random selection, you can use the following `RNG`.

```
typedef enum {left_side, right_side } SIDE;

SIDE rnd(){
    return rand()%2 ? right_side : left_side;
}// End of rnd()
```

Put the implementation of your `BinaryTree` class in the header file **btree.h**. Definition of the class `Node`, which represents the nodes in a binary tree, can be found in the header file **node.h**. To use the class `Node` in your program, include the header file **node.h**, inserting `#include "node.h"` at the top of your header file.

The source file **binarytree.cc** contains the driver program. In addition to the `main()` routine, it has the implementations of the following routines (as templates) and the definitions of the two RNGs used in the `main()` routine.

- `template<class T> void print(const T &x):`
- `template<class T> void printValues(BinaryTree<T> &tree, const string &name);`

The unary function `print()` can be used as an argument to the member functions `Inorder()` to print the value of its argument `x`. The function `printValues()` does the followings:

- it prints name, which is the name of the tree, and it also prints the height of the tree.
- it calls the member function `Inorder()` to print the data values in the tree in *inorder*.

The class `RND1` can be used to generate random integers in the range [LOW1 = –999, HIGH1 = 999] and the class `RND2` can be used to generate random floating-point numbers in the range [LOW2 = –999.99, HIGH2 = 999.99]. The function objects `RND1()` and `RND2()`, generated from these classes, are used to fill in the random values in *vector* containers `vector<int> A(N1)` and `vector<float> B(N2)` by using the `generate()` function in the STL, where `N1 = 100` and `N2 = 50` are the sizes of these two vectors.

The `main()` routine copies the random values from vectors `A` and `B` and inserts them in the binary trees `first` and `second`, respectively. At the end, the data values in the binary trees `first` and `second` are printed out on stdout with `LSIZE = 12` numbers in a single line.

The source file of the driver program **binarytree.cc**, is provided. Write a `Makefile` to compile the driver program and header files. The correct output of this program can be found in file **binarytree.refout**. **NOTE:** The *refout* assumes that a tree with **only a root is height 1**, where as others believe it is height 0, different CS professors handle this in different ways.**

Assignment Notes:

- Include any necessary headers and add necessary global constants.
- You are not allowed to use any I/O functions from the C library, such as scanf or printf. Instead, use the I/O functions from the C++ library, such as cin or cout.
- Add documentation to the appropriate source files as discussed in your class.
- Prepare your Makefile (you need to construct and add Makefile) so that the TA only needs to invoke the command make to compile your source file and produce the executable file **binarytree**. Make sure you use exactly the same file names specified here, i.e. **binarytree**, in your Makefile, otherwise your **submission will get 0 points**.

When your program is ready for grading, ***commit*** and ***push*** your local repository to remote git classroom repository and follow the ***Assignment Submission Instructions***.